Bölüm 4 Döngüler

Liang, Java Programlama'ya Giriş, Sekizinci Baskı, (c) 2011 Pearson Education, Inc. Tüm hakları saklıdır. 01321308071

Motivasyonlar

Diyelim ki bir dizeyi (örneğin, "Java'ya Hoş Geldiniz!"ı yüz kez yazdırmanız gerekiyor. Bu, şu ifadeyi yüz kez yazmak oldukça zahmetli olurdu:

```
System.out.println("Java'ya Hoş Geldiniz!");
```

Bu problemi nasıl çözersiniz?

Açılış Problemi

Problem:

100 tane

```
System.out.println("Java'ya Hoş Geldiniz!");
System.out.println("Java'ya Hoş Geldiniz!");
System.out.println("Java'ya Hoş Geldiniz!");
System.out.println("Java'ya Hoş Geldiniz!");
System.out.println("Java'ya Hoş Geldiniz!");
System.out.println("Java'ya Hoş Geldiniz!");

...

...

...
System.out.println("Java'ya Hoş Geldiniz!");
System.out.println("Java'ya Hoş Geldiniz!");
System.out.println("Java'ya Hoş Geldiniz!");
```

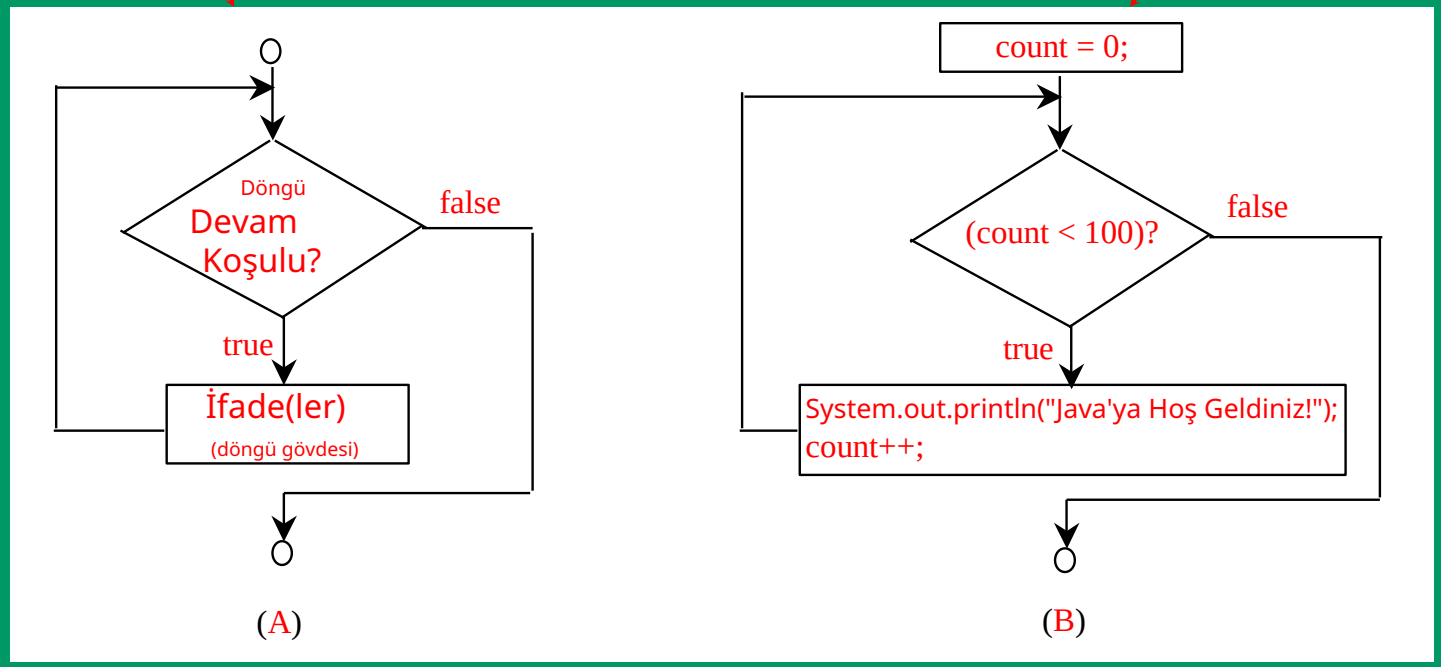
while Döngülerini Tanıtma

```
int count = 0;
while (count < 100) {
   System.out.println("Java'ya Hoş Geldiniz");
   count++;
}
```

`while` Döngü Akış Şeması

```
while (döngü-devam-etme-koşulu) {
  // döngü-gövdesi;
  İfade(ler);
}
```

```
int sayac =  0;
while (sayac <  100) {
  System.out.println("Java'ya Hoş Geldiniz!");
  sayac++;
}
```



animasyon

Trace while Döngüsü

Sayacı başlat

```
int count = 0;
while (count < 2) {
  System.out.println("Java'ya Hoş Geldiniz!");
  count++;
}
```

animasyon

Trace while Döngüsü, devam.

(count < 2) doğrudur

```
int sayac =  0;
while (count < 2) {
  System.out.println("Java'ya Hoş Geldiniz!");
  count++;
}
```

animasyon

Trace while Döngüsü, devam.

```
int count = 0;
while (count < 2) {
  System.out.println("Welcome to Java!");
  count++;
}
```

Java'ya Hoş Geldiniz Yazdır

animasyon

Trace while Döngüsü, devam.

```
int count = 0;
while (count < 2) {
  System.out.println("Java'ya Hoş Geldiniz!");
  count++;
}
```

Sayacı 1 artır

şu anda sayaç 1

animasyon

Trace while Döngüsü, devam.

```
int sayac =  0;
while (count < 2) {
  System.out.println("Java'ya Hoş Geldiniz!");
  count++;
}
```

(sayaç < 2) hala doğru çünkü sayaç 1'dir

animasyon

Trace while Döngüsü, devam.

```
int sayac =  0;
while (sayaç <  2) {
  System.out.println("Welcome to Java!");
  count++;
}
```

Java'ya Hoş Geldiniz Yazdır

animasyon

Trace while Döngüsü, devam.

```
int count = 0;
while (count < 2) {
  System.out.println("Java'ya Hoş Geldiniz!");
  count++;
}
```

Sayacı 1 artır

şu anda sayaç 2

animasyon

Trace while Döngüsü, devam.

```
int count = 0;
while (count < 2) {
  System.out.println("Java'ya Hoş Geldiniz!");
  count++;
}
```

(sayaç < 2) yanlıştır çünkü sayaç 2'dir şimdi

animasyon

Trace while Döngüsü

```
int count = 0;
while (count < 2) {
  System.out.println("Java'ya Hoş Geldiniz!");
  count++;
}
```

Döngü sona eriyor. Döngüden sonraki ifadeyi yürütün.

Problem: Sayıları Tahmin Etme

0 ile 100 arasında, dahil olmak üzere, rastgele bir tamsayı üreten bir program yazın. Program, kullanıcıdan sürekli olarak bir sayı girmesini ister ta ki girilen sayı rastgele üretilen sayıyla eşleşene kadar. Her kullanıcı girişi için, program kullanıcıya girilen sayının çok düşük mü yoksa çok yüksek mü olduğunu söyler, böylece kullanıcı bir sonraki girişi akıllıca seçebilir.

İşte bir örnek çalışma:

GuessNumberOneTime

Çalıştır

Sayıyüret

Çalıştır

Bir Döngüyü Gözetleyici Değeri ile Sonlandırma

Döngünün kaç kez çalıştırılacağı genellikle önceden b
elirlenmemiştir. Döngünün sonunu belirtmek içi
n girdi değeri kullanabilirsiniz. Böyle bir değere *göze
tleyici değeri* denir.

Belirsiz sayıda tam sayıyı okuyan ve toplam
ını hesaplayan bir program yazın. Girdi 0
girdinin sonunu belirtir.

GözetleyiciDeğeri

Çalıştır

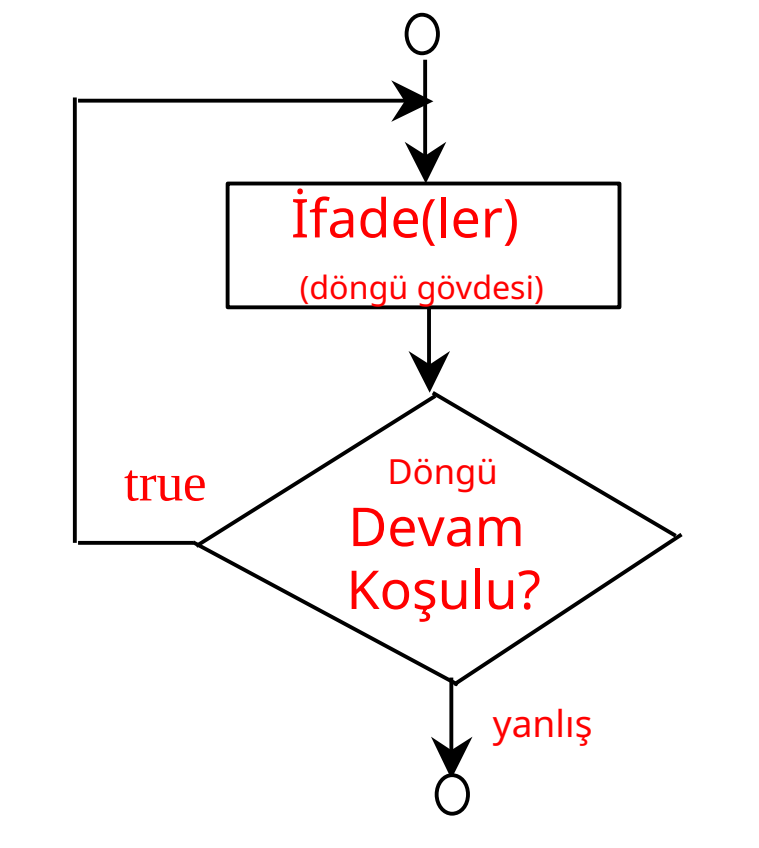
Dikkat

Döngü kontrolünde eşitlik kontrolü için kayan nokta değerleri kullanmayın.
Kesirli değerler bazı değerler için yaklaşık değerler olduğundan, bunları kullanmak belirsiz sayaç değerleri ve hatalı sonuçlar doğurabilir.
Aşağıdaki kodu 1 + 0.9 + 0.8 + ... + 0.1 hesaplamak için düşünün:

```
double item = 1; double sum = 0;
while (item != 0) { // item'in 0 olacağına dair bir garanti yok
  sum += item;
  item -= 0.1;
}
System.out.println(sum);
```

Değişken item 1 ile başlar ve döngü gövdesi her çalıştığında 0.1 azaltılır.
Döngü, item 0 olduğunda sona ermeli. Ancak,
item'in tam olarak 0 olacağına dair bir garanti yoktur, çünkü kesirli-sayı aritmetiği yaklaşık olarak hesaplanmaktadır. Bu döngü yüzeyde iyi görünüyor, ama gerçekte sonsuz bir döngüdür.

do - while Döngüsü

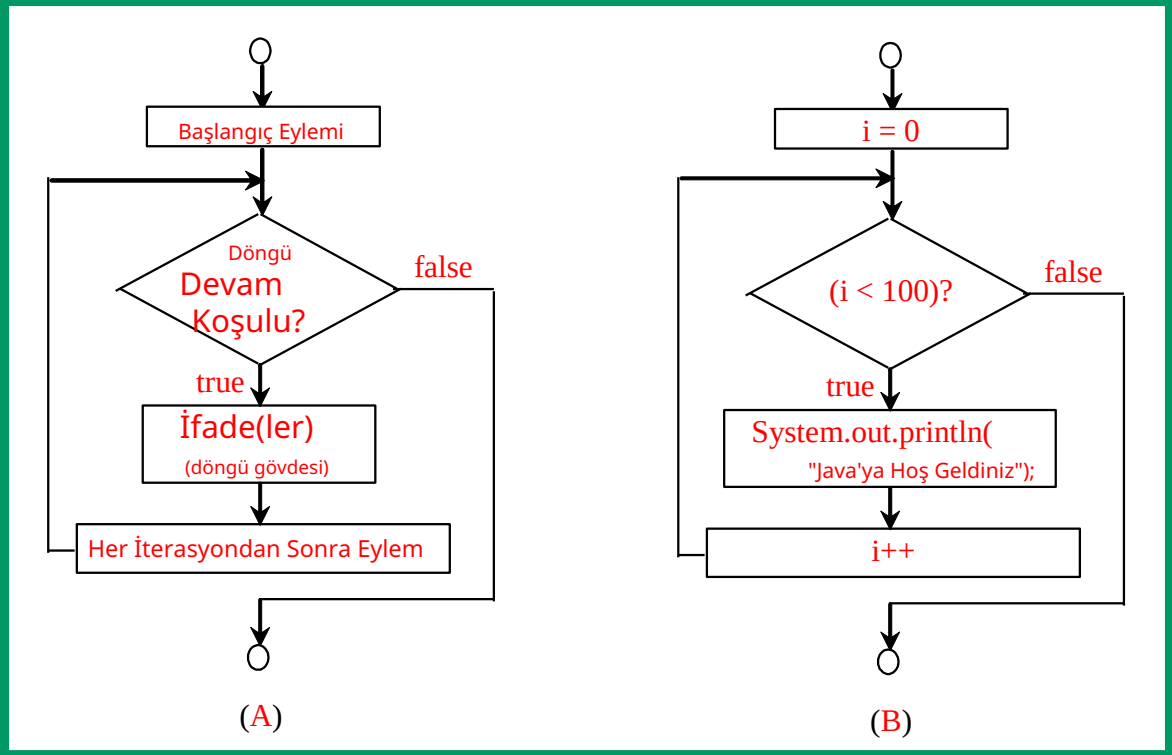



```
do {
    // Döngü gövdesi;
    İfade(ler);
} while (döngü-devam-etme-koşulu);
```

için Döngüler

```
için (başlangıç-eylemi; döngü-
    devam-etme-koşulu;
    action-after-each-iteration) {
  // döngü gövdesi;
  Açıklama(lar);
}
```

```
int i;
for (i = 0; i < 100; i++) {
  System.out.println(
    "Java'ya Hoş Geldiniz!");
}
```



animasyon

For Döngüsünü İzle

i'yi Tanımla

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println(
     "Java'ya Hoş Geldiniz!");
}
```

animasyon

For Döngüsünü İzle, devam.

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println(
     "Java'ya Hoş Geldiniz!");
}
```

Başlatıcıyı Çalıştır
i artık 0

animasyon

For Döngüsünü İzle, devam.

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println( "Java'ya Hoş Geldiniz!");
}
```

(i < 2) doğrudur
çünkü i 0'dır

animasyon

For Döngüsünü İzle, devam.

Java'ya Hoş Geldiniz Yazdır

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```

animasyon

For Döngüsünü İzle, devam.

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Java'ya Hoş Geldiniz!");
}
```

Ayarlama ifadesini çalıştır

i şimdi 1'dir

animasyon

For Döngüsünü İzle, devam.

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Java'ya Hoş Geldiniz!");
}
```

(i < 2) hala doğru

çünkü i 1'dir

animasyon

For Döngüsünü İzle, devam.

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```

Java'ya Hoş Geldiniz Yazdır

animasyon

For Döngüsünü İzle, devam.

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Java'ya Hoş Geldiniz!");
}
```

Ayarlama ifadesini çalıştır

i şimdi 2'dir

animasyon

For Döngüsünü İzle, devam.

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Java'ya Hoş Geldiniz!");
}
```

(i < 2) yanlıştır

çünkü i 2'dir

animasyon

For Döngüsünü İzle, devam.

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Java'ya Hoş Geldiniz!");
}
```

Döngüden çık. Sonraki işlemi gerçekleştir

döngüden sonraki ifade

Not

Bir döngüdeki başlangıç eylemi, sıfır veya daha fazla virgülle ayrılmış ifad elerden oluşan bir liste olabilir. Her birinden sonra yapılan eylem- bir döngüdeki yineleme, sıfır veya daha fazla virgülle ayrılmış bir liste olabilir- ayrılmış ifadeler. Bu nedenle, aşağıdaki iki döngüsü doğrudur. Pratikte nadiren kullanılır, ancak.

```
for (int i = 1; i < 100; System.out.println(i++));


for (int i = 0, j = 0; (i + j < 10); i++, j++) {
  // Bir şey yap
}
```

Not

Bir for döngüsündeki döngü devam koşulu atlandığında, it implicitly doğrudur. Böylece aşağıda (a) kısmında verilen ifade, sonsuz bir döngü olan doğrudur. Yine de, bu (b) bölümündeki eşdeğer döngüyü kullanmak, karışıklığı önlemek için daha iyidir:

```
for ( ; ; ) {
    // Bir şey yap
}
```

(a)

Eşdeğer

```
while (true) {
    // Bir şey yap
}
```

(b)

Dikkat

Bir döngü gövdesinden önceki for ifadesinin sonuna bir noktalı virgül eklemek, aşağıda gösterildiği gibi yaygın bir hatadır:

Hata

Mantığı

```
for (int i=0; i<10; i++);
{
  System.out.println("i is " + i);
}
```

Uyarı, devam.

Aynı şekilde, aşağıdaki döngü de yanlıştır:

```
int i=0;
while (i < 10);
{
  System.out.println("i is " + i);
  i++;
}
```

Mantık Hatası

do döngüsü durumunda, aşağıdaki noktalı virgül döngüyü sonlandırmak için gereklidir.

```
int i=0;
do {
  System.out.println("i is " + i);
  i++;
} while (i<10);
```

Doğru

Öneriler

En sezgisel ve rahat olanı kullanın
size.

- Genel olarak, bir for döngüsü, tekrarlama sayısı biliniyorsa kullanılabilir; örneğin, bir mesajı 100 kez yazdırmanız gere ktiğinde.
- Bir while döngüsü, tekrarlama sayısı bilinmediğinde kullanılabilir. bilinen, giriş bitene kadar sayıları okuma durumunda olduğu gibi 0.
- Bir do-while döngüsü, döngü gövdesinin devam etme koşulunu test etmeden ö nce çalıştırılması gerektiğinde bir while döngüsünün yerini almak için kullanılabilir.

İç İçe Döngüler

Problem: İç içe for kullanan bir program yazın

çarpan tablosunu yazdırmak için döngüler.

ÇarpanTablosu

Çalıştır

Problem:
En Büyük Ortak Bölgenin Bulunması

Problem: Kullanıcıdan iki pozitif

tam sayı girmesini isteyen ve bunların en büyük ortak bölenini bulan bir program yazın.

Çözüm: Diyelim ki iki tam sayı 4 ve 2 girdiniz, bunların en büyük ort ak böleni 2'dir. Diyelim ki iki tam sayı 16 ve 24 girdiniz, bunların en büyük ortak böleni 8'dir. Peki, en büyük ortak böleni nasıl bulursunuz? İki girdi tam sayısını n1 ve n2 olarak adlandıralım. 1 sayısının ortak bir böle n olduğunu biliyorsunuz, ancak bu en büyük ortak bölen olmayabilir. Bu ne denle, k'nin (k = 2, 3, 4, vb.) n1 ve n2 için ortak bir bölen olup olmadığın ı kontrol edebilirsiniz, ta ki k n1 veya n2'den büyük olana kadar.

EnBüyükOrtakBölgeninBulunması

Çalıştır

Problem: Gelecek Öğrenim Ücretini Tahmin Etme

Problem: Bir üniversitenin öğrenim ücretinin bu yıl $10,000 olduğunu ve öğrenim ücretinin her yıl %7 arttığını varsayalım. Öğrenim ücreti kaç yıl için de iki katına çıkacak?

GelecekÖğrenimÜcreti

Çalıştır

Problem: Gelecek Öğrenim Ücretini Tahmin Etme

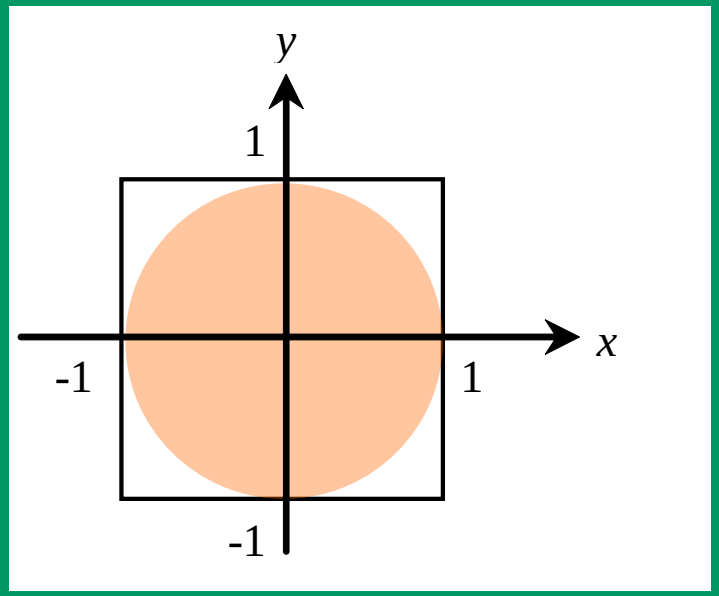
```
double öğrenim ücreti = 10000;   int yıl = 1  // Yıl 1
o öğrenim ücreti = öğrenim ücreti * 1.07; yıl++;      // Yıl 2
o öğrenim ücreti = öğrenim ücreti * 1.07; yıl++;      // Yıl 3
o öğrenim ücreti = öğrenim ücreti * 1.07; yıl++;      // Yıl 4
...
```

GelecekÖğrenimÜcreti

Çalıştır

Problem: *Monte Carlo Simülasyonu*

Monte Carlo simülasyonu, rastgele numbers ve olasılık kullanarak p roblemleri çözme tekniğini ifade eder. Bu yöntem, hesaplamalı mate matik, fizik, kimya ve finans alanlarında geniş bir uygulama ye lpazesine sahiptir. Bu bölüm, Monte Carlo simülasyonunu kullanar ak π tahmin etme örneği vermektedir.



daireAlanı / kareAlanı = π/ 4.

π yaklaşık olarak 4 * numberOfHits / 1000000.

MonteCarloSimülasyonu

Çalıştır

Using `break` ve `continue`

break ve continue kullanımı için örnekler

anahtar kelimeler:

- TestBreak.java

TestBreak

Çalıştır

- TestContinue.java

TestContinue

Çalıştır

Sayısal Tahmin Problemi Yeniden İncelendi

İşte bir sayı tahmin etme programı. Yapabilirsiniz break ifadesi kullanarak yeniden yazın.

GuessNumberUsingBreak

Çalıştır

(GUI) Bir Döngüyü Kontrol Etmek için Bir Onay Diyaloğu

Bir bekçi kontrollü döngü, bir onay kullanılarak uygulanabilir. dialog. Döngüyü devam ettirmek veya sonlandırmak için cevaplar *Evet* veya *Hayır*. Döngünün şablonu aşağıdaki gibi görünebilir:

```
int option = 0;
while (option == JOptionPane.YES_OPTION) {
  System.out.println("continue loop");
  option = JOptionPane.showConfirmDialog(null, "Devam et?");
}
```

SentinelValueUsingConfirmationDialog

Çalıştır